Séance 04 - Correction et guide enseignant

Table of Contents

1. Intentions pédagogiques . . . 1
2. Points de vigilance . . . 2
3. Correction - Réactivation . . . 2
4. Correction - Activité 1 . . . 2
5. Correction - Activité 2 . . . 2
6. Correction - Activité 3 . . . 3
7. Correction - Activité 4 . . . 3
   7.1. Trace . . . 3
   7.2. Services . . . 3
8. Correction - Activité 5 . . . 4
9. Correction - Activité 6 . . . 4
10. Correction - Activité 7 . . . 4
11. Correction - Activité 8 . . . 5
12. Correction - Exercice autonome . . . 5
13. Correction - Travail sur papier . . . 6
14. Correction - Synthèse . . . 7
15. Correction - Exercices bonus . . . 7
    15.1. Exercice bonus 1 [*] - Le serveur le plus chargé . . . 7
    15.2. Exercice bonus 2 [**] - Amplitude des charges . . . 8
    15.3. Exercice bonus 3 [*] - Défi : un rapport qui ne plante pas . . . 9
16. Validation Git attendue . . . 10
17. Évolution par rapport au support archivé . . . 10

1. Intentions pédagogiques

Cette version reprend le choix pédagogique du TP 3 archivé : après une introduction minimale des tableaux, le parcours se fait principalement avec `foreach`.

`for` et `while` ne font l'objet que d'un aperçu. Le cœur de la séance est :

1. tableau indexé et tableau associatif ;
2. inspection avec `print_r()` et comptage avec `count()` ;
3. parcours des valeurs ;
4. parcours clé-valeur ;

5. condition pendant le parcours ;
6. compteur, accumulateur et moyenne ;
7. diagnostic d'erreurs propres à `foreach`.

Les tableaux multidimensionnels, les références, la modification en place, `break`, `continue` et les fonctions sont différés.

2. Points de vigilance

- Employer des noms au singulier pour la variable représentant l'élément courant.
- Faire verbaliser tableau, clé et valeur avant l'exécution.
- Distinguer ⇒ de `>=`.
- Montrer que `foreach` gère lui-même le passage à l'élément suivant et l'arrêt.
- Faire prévoir les traces avant d'afficher le résultat.
- Initialiser compteurs et accumulateurs avant la boucle.
- Ne pas transformer la séance en étude exhaustive des tableaux : seuls les éléments nécessaires au parcours sont introduits.

3. Correction - Réactivation

1. `true`
2. `if/elseif/else`
3. `+` ou `= 1`
4. `git status`
5. `docker compose ps`

4. Correction - Activité 1

```
git status
git pull
docker compose up -d
```

`git status` est vérifié avant `git pull` afin de repérer un travail local non enregistré. Le serveur se lance depuis le dossier du TP ; contrôle sur `http://localhost:8181/`.

5. Correction - Activité 2

Besoin	Boucle
Afficher les numéros de `1` à `12`	`for`

Besoin	Boucle
Retenter une opération tant qu'elle échoue	`while`
Afficher tous les serveurs d'un tableau	`foreach`
Traiter toutes les notes associées à des étudiants	`foreach`

Une limite de tentatives devrait être ajoutée dans une véritable opération réessayée afin d'éviter une boucle infinie.

6. Correction - Activité 3

Après l'ajout, `$serveurs` contient quatre éléments indexés de `0` à `3`. `count($serveurs)` renvoie `4`.

Dans `$chargesCpu`, les clés sont des noms significatifs plutôt que des positions numériques. L'expression `$chargesCpu["srv-web-01"]` accède directement à la valeur `45`.

La différence essentielle :

- `0` désigne une position générée automatiquement ;
- `"srv-web-01"` est une clé choisie pour identifier la donnée.

7. Correction - Activité 4

7.1. Trace

Répétition	`$serveur`	Texte
1	`"srv-web-01"`	`Vérification de srv-web-01`
2	`"srv-bdd-01"`	`Vérification de srv-bdd-01`
3	`"srv-cache-01"`	`Vérification de srv-cache-01`

`foreach` s'arrête après le dernier élément du tableau. Le développeur n'écrit ni indice, ni condition d'arrêt, ni évolution.

7.2. Services

```
<?php

$services = ["Apache", "PostgreSQL", "Redis", "Mailpit"];

foreach ($services as $service) {
    echo "Service supervisé : " . $service . "<br>";
}
```

8. Correction - Activité 5

```php
<?php

$adressesIp = [
    "srv-web-01" => "192.168.10.11",
    "srv-bdd-01" => "192.168.10.12",
    "srv-cache-01" => "192.168.10.13"
];

foreach ($adressesIp as $nomServeur => $adresseIp) {
    echo "Le serveur " . $nomServeur . " utilise l'adresse " . $adresseIp . ".<br>";
}
```

Un parcours des seules valeurs fournirait les adresses, mais pas les noms des serveurs. La syntaxe clé-valeur conserve les deux informations.

9. Correction - Activité 6

Serveur	État
`srv-web-01`	`NORMAL`
`srv-bdd-01`	`CRITIQUE`
`srv-cache-01`	`NORMAL`
`srv-fichiers-01`	`SURVEILLANCE`

La structure conditionnelle est évaluée une fois pour chaque couple clé-valeur, donc quatre fois.

10. Correction - Activité 7

Serveur traité	Serveurs	Alertes	Total
`srv-web-01`	`1`	`0`	`45`
`srv-bdd-01`	`2`	`1`	`137`
`srv-cache-01`	`3`	`1`	`165`
`srv-fichiers-01`	`4`	`2`	`241`

La charge moyenne vaut `241 / 4`, soit `60.25`.

Les variables sont initialisées avant la boucle pour conserver les résultats entre les répétitions. `$totalCharges` est l'accumulateur. `$nombreServeurs` fournit le diviseur. La moyenne ne peut être calculée qu'une fois le total et le nombre définitifs connus.

`count($chargesCpu)` peut remplacer `$nombreServeurs`, mais le compteur explicite est conservé pour travailler la trace et généraliser aux comptages conditionnels.

11. Correction - Activité 8

Le fichier fourni comporte trois erreurs logiques.

Code fautif	Correction	Explication
`print_r($services)`	afficher `$service`	La variable d'élément contient le service courant.
`foreach ($chargesCpu as $chargeCpu)`	`foreach ($chargesCpu as $nomServeur ⇒ $chargeCpu)`	La clé est nécessaire pour afficher le nom du serveur.
`if ($chargeCpu < 70)`	`if ($chargeCpu >= 70)`	Le compteur doit compter les alertes, pas les situations normales.

Correction complète :

```
<?php

$services = ["Apache", "PostgreSQL", "Redis"];

foreach ($services as $service) {
    echo "Service : " . $service . "<br>";
}

echo "<hr>";

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 76
];

$nombreAlertes = 0;

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    echo $nomServeur . " : " . $chargeCpu . " %<br>";

    if ($chargeCpu >= 70) {
        $nombreAlertes++;
    }
}

echo "Nombre d'alertes : " . $nombreAlertes;
```

12. Correction - Exercice autonome

```php
<?php

$espacesLibres = [
    "srv-web-01" => 42,
    "srv-bdd-01" => 8,
    "srv-cache-01" => 65,
    "srv-fichiers-01" => 12,
    "srv-sauvegarde-01" => 5
];

$nombreServeurs = 0;
$nombreCritiques = 0;
$espaceLibreTotal = 0;

foreach ($espacesLibres as $nomServeur => $espaceLibreGo) {
    $nombreServeurs++;
    $espaceLibreTotal += $espaceLibreGo;

    if ($espaceLibreGo < 10) {
        $etat = "CRITIQUE";
        $nombreCritiques++;
    } elseif ($espaceLibreGo < 20) {
        $etat = "SURVEILLANCE";
    } else {
        $etat = "NORMAL";
    }

    echo $nomServeur . " : " . $espaceLibreGo . " Go - " . $etat . "<br>";
}

$espaceLibreMoyen = $espaceLibreTotal / $nombreServeurs;

echo "Serveurs : " . $nombreServeurs . "<br>";
echo "Serveurs critiques : " . $nombreCritiques . "<br>";
echo "Espace libre total : " . $espaceLibreTotal . " Go<br>";
echo "Espace libre moyen : " . $espaceLibreMoyen . " Go";
```

Résultats :

- 5 serveurs ;
- 2 serveurs critiques ;
- 132 Go libres au total ;
- 26.4 Go libres en moyenne.

13. Correction - Travail sur papier

$service	$numeroPort	$total
HTTP	80	80
HTTPS	443	523
SSH	22	545

Texte affiché :

```
HTTP : 80
HTTPS : 443
SSH : 22
Total : 545
```

Avec `foreach ($ports as $numeroPort)`, la variable reçoit successivement `80`, `443` et `22`.

Pour afficher uniquement les clés, il faut tout de même déclarer une variable pour la valeur :

```
foreach ($ports as $service => $numeroPort) {
    echo $service . "<br>";
}
```

14. Correction - Synthèse

1. tableau
2. `0`
3. tableau associatif
4. `print_r()`
5. `count()`
6. `foreach`
7. la valeur courante
8. la clé courante
9. `⇒`
10. accumulateur

15. Correction - Exercices bonus

15.1. Exercice bonus 1 [*] - Le serveur le plus chargé

```
<?php

$chargesCpu = [
```

```
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 28,
    "srv-fichiers-01" => 76
];

$nomLePlusCharge = "";
$chargeLaPlusHaute = -1;

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    if ($chargeCpu > $chargeLaPlusHaute) {
        $chargeLaPlusHaute = $chargeCpu;
        $nomLePlusCharge = $nomServeur;
    }
}

echo "Le plus chargé : " . $nomLePlusCharge . " (" . $chargeLaPlusHaute . " %)";
```

Sortie : `Le plus chargé : srv-bdd-01 (92 %)`. Point clé : mettre à jour **deux** variables ensemble (la valeur maximale et le nom associé). L'initialisation à `-1` garantit que le premier serveur passe le test.

15.2. Exercice bonus 2 [**] - Amplitude des charges

```
<?php

$chargesCpu = [
    "srv-web-01" => 45,
    "srv-bdd-01" => 92,
    "srv-cache-01" => 28,
    "srv-fichiers-01" => 76
];

$chargeMin = null;
$chargeMax = null;
$nomMin = "";
$nomMax = "";

foreach ($chargesCpu as $nomServeur => $chargeCpu) {
    if ($chargeMin === null || $chargeCpu < $chargeMin) {
        $chargeMin = $chargeCpu;
        $nomMin = $nomServeur;
    }

    if ($chargeMax === null || $chargeCpu > $chargeMax) {
        $chargeMax = $chargeCpu;
        $nomMax = $nomServeur;
    }
}
```

```
echo "Min : " . $nomMin . " (" . $chargeMin . " %)<br>";
echo "Max : " . $nomMax . " (" . $chargeMax . " %)<br>";
echo "Amplitude : " . ($chargeMax - $chargeMin) . " points";
```

Sortie : min `srv-cache-01 (28 %)`, max `srv-bdd-01 (92 %)`, amplitude `64 points`. L'initialisation à `null` évite de choisir une borne arbitraire : le premier tour fixe les deux mémoires. Un seul parcours suffit pour les deux extrêmes.

15.3. Exercice bonus 3 [*] - Défi : un rapport qui ne plante pas

Sur un tableau vide, le calcul `$totalCharges / $nombreServeurs` divise par zéro. En PHP 8, cela lève :

```
Fatal error: Uncaught DivisionByZeroError: Division by zero
```

Version protégée :

```
<?php

$chargesCpu = []; // tester d'abord vide, puis avec des charges

$nombreServeurs = 0;
$totalCharges = 0;

foreach ($chargesCpu as $chargeCpu) {
    $nombreServeurs++;
    $totalCharges += $chargeCpu;
}

if ($nombreServeurs === 0) {
    echo "Aucun serveur à superviser.";
} else {
    echo "Charge moyenne : " . ($totalCharges / $nombreServeurs) . " %";
}
```

Sur `[]` : `Aucun serveur à superviser.` ; sur le tableau de charges rempli : `Charge moyenne : 60.25 %`.

Guide enseignant : c'est un premier réflexe de **robustesse** — vérifier le cas limite (parc vide) avant une division. On peut relier cela au `count()` de la synthèse : `count($chargesCpu)` vaut `0` sur un tableau vide. Insister sur le fait qu'un programme correct sur les données « normales » peut échouer sur une donnée limite.

16. Validation Git attendue

```
Ajoute les traitements avec foreach
Ajoute le parcours des tableaux indexés
```

Après `git push`, `git status` doit indiquer un espace de travail propre.

17. Évolution par rapport au support archivé

La séance conserve du TP 3 archivé :

- la distinction entre tableau indexé et associatif ;
- `print_r()` dans des balises `<pre>` ;
- `count()` ;
- le parcours `foreach` des valeurs ;
- le parcours `foreach` clé-valeur.

Elle développe davantage :

- la lecture détaillée de la syntaxe ;
- les traces d'exécution ;
- les noms de variables ;
- l'association de conditions au parcours ;
- les compteurs, accumulateurs et moyennes ;
- le diagnostic d'erreurs ;
- un exercice autonome contextualisé.

Le `for` utilisé dans le bonus de l'ancien support pour fusionner deux tableaux n'est pas repris : il détournerait l'attention du parcours avec `foreach`.